

FIAP
2TIAPF 2026

AUGUSTO OLIVEIRA CODO DE SOUSA
FELIPE DE OLIVEIRA CABRAL
GABRIEL TONELLI AVELINO DOS SANTOS
VINÍCIUS ADRIAN SIQUEIRA DE OLIVEIRA
SOFIA BUERIS NETTO DE SOUZA

DOCUMENTO DE JUSTIFICATIVA TÉCNICA

Assistente Virtual com Arquitetura RAG

Trabalho acadêmico apresentado à disciplina de
Generative AI Advanced Net como requisito de
avaliação.

SÃO PAULO
2026

1. ESCOLHA DO BANCO DE DADOS VETORIAL

A equipe adotou o **Qdrant** como banco de dados vetorial, provisionado localmente via **Docker Compose**. A decisão fundamenta-se em três pilares técnicos: (i) o Qdrant implementa o algoritmo de indexação **HNSW** (*Hierarchical Navigable Small World*), que garante buscas aproximadas de vizinhos mais próximos em complexidade sublogarítmica — viabilizando recuperação semântica de alta performance mesmo em coleções com milhões de vetores; (ii) a containerização via Docker elimina dependências de ambiente, assegurando reprodutibilidade idêntica entre as estações de trabalho dos membros da equipe e em ambientes de produção futuros; e (iii) a biblioteca oficial `qdrant-client` oferece integração nativa com Python, expondo uma API assíncrona compatível com o paradigma do FastAPI. A métrica de similaridade **Cosine Distance** foi configurada para a coleção, sendo a escolha canônica para espaços de embeddings de linguagem, onde a magnitude do vetor é irrelevante e apenas a orientação semântica importa.

2. ESCOLHA DO MODELO DE LINGUAGEM E EMBEDDINGS

Para a camada de inferência de linguagem natural, a equipe substituiu a integração padrão com a OpenAI pela combinação estratégica de dois serviços complementares:

2.1 SENTENCE TRANSFORMERS — ALL-MINILM-L6-V2 (EMBEDDINGS LOCAIS)

O processo de vetorização de documentos e consultas é executado integralmente de forma *offline*, sem transmissão de dados a serviços externos. Esta decisão garante privacidade irrestrita dos dados indexados, elimina latência de rede na fase de recuperação semântica e zera custos de API para esta operação. O modelo `all-MiniLM-L6-v2` gera vetores de **384 dimensões**, oferecendo um equilíbrio ótimo entre qualidade semântica e eficiência computacional, sendo executável em hardware de propósito geral sem GPU dedicada.

2.2 GROQ CLOUD API — MODELO LLAMA3-8B-8192 (INFERÊNCIA DE TEXTO)

Para a geração de respostas conversacionais, optou-se pela Groq Cloud API, que utiliza hardware LPU (*Language Processing Unit*) proprietário. Esta arquitetura proporciona velocidades de inferência significativamente superiores às APIs

convencionais baseadas em GPU, com latências médias abaixo de 500ms para respostas completas. O modelo Llama 3, desenvolvido pela Meta AI e disponibilizado na modalidade de acesso gratuito pela Groq, apresenta desempenho comparável a modelos proprietários de grande porte em tarefas de compreensão e geração de linguagem em português.

3. MELHORIAS DE DESIGN IMPLEMENTADAS

3.1 Estratégia de Chunking com Sobreposição (Overlap)

O pipeline de ingestão de documentos (`scripts/index_documents.py`) implementa um *Text Splitter* baseado em janela deslizante com parâmetros de **1.000 caracteres por chunk** e **200 caracteres de sobreposição**. A sobreposição é tecnicamente necessária para preservar a coerência semântica em fronteiras de chunk: sentenças ou conceitos partidos no limite de um chunk permanecem representados integralmente no início do chunk subsequente, evitando degradação da qualidade na recuperação semântica. Esta abordagem maximiza a relevância dos documentos retornados pelo Qdrant para cada consulta.

3.2 Separação de Responsabilidades e Injeção de Dependência

A arquitetura do back-end segue rigorosamente os princípios **SOLID**, em especial o *Dependency Inversion Principle*. O serviço `GroqRAGService` herda da classe abstrata `BaseLLMService(ABC)`, garantindo que o `ChatOrchestrator` dependa exclusivamente de abstrações, não de implementações concretas. No `FastAPI`, os serviços são injetados via `Depends()` com sobreposição por `dependency_overrides`, permitindo substituição de provedores de IA sem alteração das rotas e possibilitando testes unitários com *mocks* isolados. Os serviços pesados (`SentenceTransformer`, `QdrantClient`) são instanciados uma única vez no `lifespan` da aplicação, eliminando recarregamentos desnecessários por requisição.

3.3 Segurança e Gestão de Configurações

Todas as credenciais e parâmetros sensíveis — incluindo `GROQ_API_KEY`, `QDRANT_API_KEY` e demais configurações operacionais — são lidos exclusivamente através de variáveis de ambiente gerenciadas pelo `pydantic-settings`. A classe `Settings` implementa um `field_validator` que aborta a inicialização da aplicação caso a chave de API esteja ausente ou contenha o valor padrão fictício, tornando o erro de configuração explícito e imediato. O arquivo `.env`

contendo valores reais está listado no `.gitignore`, enquanto o `.env.example` com valores fictícios é versionado no repositório — permitindo onboarding seguro de novos membros da equipe sem exposição de credenciais.

Documento elaborado pela Equipe Predicta — FIAP 2026